Egypt-Japan University of Science and Technology
(E-JUST)

AI-Based Fruit Spoilage Detection

Report 6 - AI + Hardware System Integration

Course: ECE 315 – Seminar and Project Based Learning
Semester: Fall 2026 — Week 6 (26/10/2026)
Group: ECE 1

Student Name	Student ID
Bashar Ahmed	120240009
Mohamed Ahmed Galal	120240122
Mohamed Tamer	120240111
Sara Saber	120240100
Mohamed Mostafa	120240184
Abdelrahman Hisham	120240076

Course Instructor: Prof. Adel Bedair

Table of Contents

ABSTRACT **3**
1. INTRODUCTION & WEEK 6 OBJECTIVES **3**
1.1 Overview of Week 6 Progress 3
1.2 Primary Objectives for System Integration 3
2. LITERATURE REVIEW **3**
2.1 Comparison with Prior Solution 4
2.2 How Design Improves the Limitations of the Prior Solution 5
2.3 References Used in the Literature Review 5
3. INTEGRATED HARDWARE ARCHITECTURE & SENSOR INTERFACING **6**
4. AI MODEL ARCHITECTURE & TRAINING EVALUATION **7**
4.1 Multi-Task Convolutional Neural Network (CNN) Setup 7
4.2 Training Performance & Test Metrics 7
5. FIRMWARE, TELEMETRY PIPELINE & SERVER-SIDE INFERENCE **7**
5.1 End-to-End Pipeline 7
5.2 Acquisition Cycle and Message Flow 8
5.3 Payload Format 8
5.4 Firmware Behaviour 9
6. SENSOR-VISION FUSION LOGIC **9**
7. INTEGRATION TESTING & RESULTS **9**
7.1 Expected Telemetry Trends 10
8. TECHNICAL CHALLENGES, ROOT CAUSE ANALYSIS & APPLIED SOLUTIONS **10**
9. TEAM TASK DISTRIBUTION & PROGRESS ALIGNMENT **11**
10. WEEK 6 OUTCOME & BRIEF DESCRIPTION OF REPORT 7 ..... **11**
10.1 Week 6 Outcome 11
10.2 Brief Description of Report 7 11
11. REFERENCES **11**

ABSTRACT

This report presents the Week 6 integration of the trained AI model with the sensing hardware of the AI-based fruit spoilage detection system. In Report 5, a sealed enclosure prototype combining an ESP32-CAM, a DHT22 temperature and humidity sensor and an MQ135 gas sensor was assembled, and a multi-task convolutional neural network (CNN) with a shared four-block feature extractor and two classification heads was trained, reaching 92.3% validation produce accuracy, 95.1% validation freshness accuracy and an 89.4% joint exact-match rate on the held-out test set. In Week 6, these parts were joined into one end-to-end pipeline: the ESP32-CAM sends each JPEG frame and its sensor readings in a single multipart HTTP request to a local inference server, which runs the CNN and a sensor-vision fusion stage in which the detected produce type selects a produce-specific gas limit, and returns a state of FRESH, EARLY WARNING or SPOILED. The report reviews the disadvantages of existing monitoring approaches, benchmarks the design against an IoT gas-sensing baseline, and describes the hardware layout, model, firmware, payload format and fusion logic, together with the integration challenges and the solutions applied. System-level measurements such as delivery rate, latency and uptime were not recorded during Week 6 and are scheduled for Week 9.

Index Terms—fruit spoilage detection, ESP32-CAM, multi-task CNN, sensor-vision fusion, MQ135, DHT22, edge-to-server inference.

1. INTRODUCTION & WEEK 6 OBJECTIVES

1.1 Overview of Week 6 Progress

In accordance with the parallel project schedule, Week 6 is the joint AI + Hardware System Integration milestone for both the hardware and software tracks. During this week, the trained multi-task CNN [1] and the sensing hardware [2], [3], [4] were connected into a single end-to-end pipeline: the ESP32-CAM acquires an image and environmental readings inside the enclosure, transmits them over Wi-Fi to a local inference server, and receives a freshness decision that combines the CNN output with gas-sensor evidence.

1.2 Primary Objectives for System Integration

- Define a single JSON/JPEG payload format that carries image data and sensor telemetry in one HTTP POST request, so that every image is paired with readings taken at the same moment (multipart encoding as specified in [5]).
- Deploy the trained PyTorch model behind a local inference server that preprocesses incoming frames, runs the multi-task CNN and returns the result to the edge node.
- Implement a sensor-vision fusion stage in which the detected produce type selects a produce-specific gas limit, addressing the main weakness of threshold-only systems identified in the literature review [6].
- Verify reliability of the full pipeline under continuous operation in the sealed enclosure, including Wi-Fi interruptions and power transients.

2. LITERATURE REVIEW

Prior work on fruit quality monitoring falls into several families of approaches: machine-vision inspection, gas (electronic-nose) sensing, temperature/humidity monitoring, edge or cloud AI inference, and multi-task learning. Each offers clear benefits but also carries limitations that affect a low-cost, continuous monitoring system. Table 1 lists the disadvantage reported for each family and the measure through which it was addressed in this project. Section 2.1 then benchmarks the system against a specific IoT baseline, and Section 2.2 describes how its limitations were overcome.

Table 1. Disadvantages of existing approaches and how each was solved in this project.

Approach	Key Disadvantage	Why It Matters	Solution Applied in This Project
Machine-vision inspection only [7], [8]	Sees only surface changes; sensitive to lighting and to the gap between public datasets and live camera images.	Early internal decay appears before visible damage; changing room light causes false classifications.	Fixed camera geometry, identical preprocessing for training and inference, and auto-exposure settled before capture; gas sensing added as a second modality.
Gas / electronic-nose sensing (MQ-series) [9], [4]	Cross-sensitivity, baseline drift, warm-up time and humidity dependence.	Raw readings change with conditions and are hard to compare between sessions.	Warm-up period before use, per-session baseline R0 with ratio G = R/R0, and humidity from the DHT22 logged alongside every gas value.
Temperature / humidity monitoring only [3]	Indicates risk conditions, not the actual state of the fruit.	Fruit may stay fresh in poor conditions or spoil in good ones.	DHT22 data used as context and logged as a feature; the freshness decision is made by the CNN and fusion rules.
Edge-only inference on a microcontroller [10], [11]	Very limited RAM, flash and compute; accuracy loss after heavy compression.	A 224×224 CNN does not fit comfortably on the ESP32-CAM beside the frame buffers and Wi-Fi stack.	Inference runs on a local server; the ESP32-CAM only acquires and transmits. INT8 quantization is evaluated separately (planned for Week 11).
Cloud-based inference [8]	Network latency, recurring cost, privacy exposure and dependence on internet access.	Storage or retail rooms may have unreliable connectivity.	Local-network inference server; firmware retries with exponential back-off and results are logged locally.
Separate single-task CNNs [1]	Two models are needed for produce type and freshness, doubling memory and compute.	Duplicate backbones increase latency and deployment effort.	One multi-task CNN with a shared feature extractor and two heads gives both outputs in a single forward pass.

As presented in Table 1, each family of prior work has a disadvantage that matters for a low-cost, continuously running monitor, and each disadvantage maps to a specific design decision in the integrated system.

2.1 Comparison with Prior Solution

The system was benchmarked against an IoT fruit-monitoring system from the literature (Bhattacharjee *et al.*, 2025) [6]. The prior system tracks spoilage with gas and microclimate sensors, while this project combines embedded computer vision, gas sensing and temperature/humidity monitoring. Table 2 gives a feature-by-feature comparison.

Table 2. Systematic comparison of the prior baseline and the proposed multi-modal system.

Feature / Dimension	Prior Baseline: Bhattacharjee et al. [6]	Proposed System (ECE 315)
Sensing modality	Gas array (MQ series) + DHT temperature/humidity	Multi-modal: OV2640 camera + MQ135 + DHT22
Detection scope	Air/gas emissions in the container only	Dual-domain: visual surface defects + headspace gas
Decision logic	Static mathematical thresholds	Multi-task CNN (produce ID + freshness) with fusion rules
Hardware platform	ESP32 microcontroller (built-in Wi-Fi)	ESP32-CAM (OV2640 + 4 MB PSRAM)
Test setup	Sealed transparent container (gas only)	Sealed, optically shielded enclosure
Produce awareness	Banana only; cannot identify produce	Automatic recognition (92.3% validation accuracy)

As presented in Table 2, the baseline relies on gas thresholds for a single fruit, while the proposed system adds a camera, recognises the produce type and combines both kinds of evidence in its decision.

2.2 How Design Improves the Limitations of the Prior Solution

The prior system provided useful guidance for sensor selection, but its design exposed several engineering bottlenecks. Each limitation and the corresponding improvement are described below.

1. Solving "blind" thresholds with automatic fruit recognition

- *The problem:* the prior system measures gas levels without knowing which fruit is being tested. Bananas release much more ethylene during natural ripening than oranges, so static thresholds cause frequent false alarms [6], [9].
- *The fix:* a dual-head CNN identifies the fruit type (92.3% validation accuracy) alongside the freshness state (95.1% validation accuracy), which allows crop-specific gas limits instead of one rigid threshold (Section 6).

2. Detecting surface blemishes through dual-domain redundancy

- *The problem:* gas sensors alone cannot detect localized bruising, surface mold patches or skin cuts while the overall gas concentration stays below the trigger limit [6].
- *The fix:* pairing the OV2640 camera with the MQ135 sensor covers both failure modes. The camera detects visible surface defects immediately, while the gas sensor catches hidden internal decomposition that is not yet visible.

3. Preventing gas dissipation through a sealed enclosure (Fig. 1)

- *The problem:* in open or drafty environments, air currents disperse volatile gases quickly, causing delayed or missed readings [9].
- *The fix:* the setup was placed in a sealed acrylic container, which lets ethylene and VOCs build up to measurable levels while the shielded walls protect the camera from changing room light and shadows.

4. Eliminating sensor brownouts and timing contention

- *The problem:* the high current draw of the MQ135 heating element (~150 mA) and improper polling intervals cause microcontroller reboots and read errors [3], [4].
- *The fix:* the MQ135 heater was moved onto a dedicated regulated 5 V rail to protect the ESP32 from voltage drops, and a non-blocking 2000 ms polling timer in firmware removed DHT22 NaN read errors.

2.3 References Used in the Literature Review

Table 3 lists the sources cited in this section, the topic each covers, and the part of the project design it informed. Full citations are given in Section 11.

Table 3. References used in the literature review and their contribution.

Ref.	Source	Topic Covered	How It Was Used in This Project
[7]	Cubero et al., Food Bioprocess Technol., 2011	Machine vision for inspection and grading of fruit and vegetables	Basis for the vision-only row of Table 1.
[8]	Kamilaris and Prenafeta-Boldú, Comput. Electron. Agric., 2018	Survey of deep learning in agriculture	Supports CNN-based classification and the cloud-inference trade-offs in Table 1.
[6]	Bhattacharjee et al., 2025	IoT food monitoring with MQ-series gas and microclimate sensors	Benchmark system compared in Section 2.1; its limitations drive Section 2.2.
[1]	Caruana, Mach. Learn., 1997	Multitask learning	Justifies one shared feature extractor with two heads.
[10]	Jacob et al., CVPR 2018	Integer-only quantization of neural networks	Supports the INT8 quantization plan and the accuracy trade-off of edge deployment.
[9]	Gardner and Bartlett, Sens. Actuators B, 1994	Electronic noses and gas-sensor arrays	Background on cross-sensitivity and baseline drift.
[11]	Howard et al., MobileNets, 2017	Lightweight CNN architectures	Reference for compact models on constrained devices.

Ref.	Source	Topic Covered	How It Was Used in This Project
[3]	Aosong DHT22 manual, 2021	Temperature and humidity sensor specification	Sensor limits and the 2000 ms minimum sampling interval.
[4]	MQ135 datasheet	Gas sensor characteristics	Heater current, warm-up and humidity sensitivity behind the baseline-ratio approach.

As presented in Table 3, every source cited in the literature review is linked to the part of the design it informed, from the choice of sensors to the model architecture and the deployment strategy.

3. INTEGRATED HARDWARE ARCHITECTURE & SENSOR INTERFACING

The physical enclosure maintains a sealed atmosphere around the fruit specimens, allowing the gas and environmental sensors to detect volatile organic compounds (VOCs) and relative humidity changes while the camera keeps an unobstructed view. The integrated hardware layout is shown in Fig. 1. The central ESP32-CAM [2] reads the DHT22 [3] and the MQ135 [4], captures the image from the on-board OV2640 camera and transmits both over Wi-Fi. Pin assignments are listed in Table 4; specific GPIO pins were chosen to prevent bus conflicts between the camera parallel data lines and the peripheral lines.

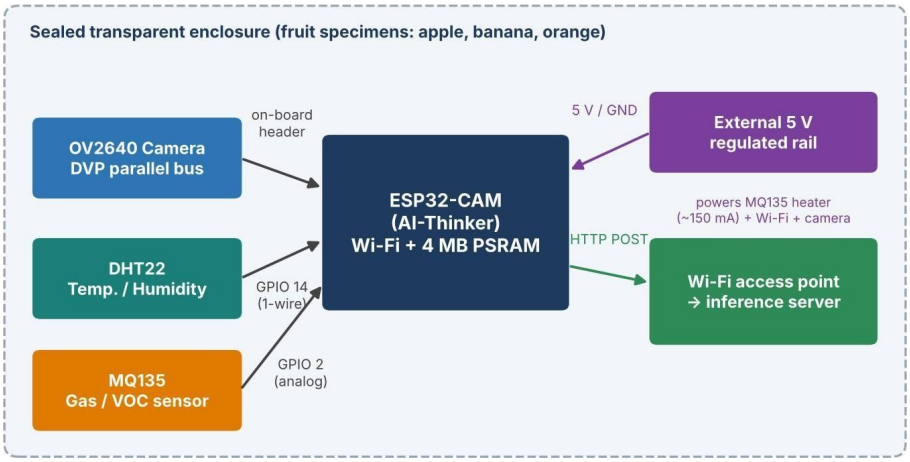


Fig. 1. Integrated hardware layout: sealed enclosure, ESP32-CAM, sensors and external 5 V rail.

As shown in Fig. 1, the ESP32-CAM is the central node of the sealed enclosure: it receives images from the OV2640 through the on-board header, temperature and humidity from the DHT22 on GPIO 14, and the MQ135 gas signal on GPIO 2. An external 5 V regulated rail powers the node, and the collected data is sent by HTTP POST through the Wi-Fi access point to the inference server.

Table 4. Hardware pin assignment and module functionality.

Module / Sensor	Connected Pin on ESP32-CAM	Protocol / Signal	Purpose & Functionality
OV2640 Camera	Dedicated camera headers	DVP parallel bus	Captures the fruit image sent to the CNN.
DHT22 Sensor	GPIO 14	1-wire digital	Ambient temperature and relative humidity monitoring [3].
MQ135 Gas Sensor	GPIO 2 / ADC channel	Analog voltage	Detects VOCs and ethylene-related gas changes [4].
Power Supply	5 V / GND pins	Regulated DC rail	Stable continuous current for the gas heater, camera and Wi-Fi.

As presented in Table 4, the camera uses its dedicated header, the two sensors share the remaining GPIO pins, and a separate regulated rail supplies the current needed by the gas heater, camera and Wi-Fi radio.

4. AI MODEL ARCHITECTURE & TRAINING EVALUATION

4.1 Multi-Task Convolutional Neural Network (CNN) Setup

A custom multi-task CNN implemented in PyTorch [12] was selected for deployment, following the multi-task learning principle of sharing one representation between related tasks [1]. As shown in Fig. 2, the model uses a shared convolutional feature extractor with four sequential blocks (channel widths 32, 64, 128 and 256). The resulting features feed two parallel fully connected heads: one for produce category (apple, banana, orange) and one for binary freshness (fresh vs. rotten). Training used the Fresh and Rotten fruit dataset [13] with the cleaning, augmentation and 80/20 split described in Report 4.

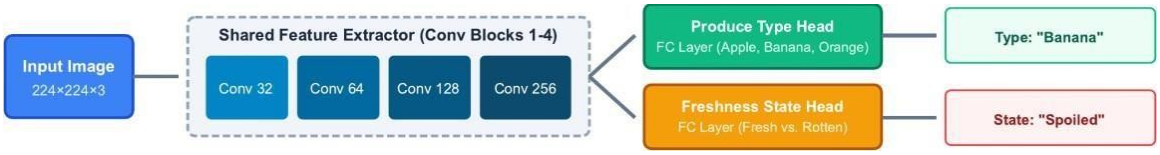


Fig. 2. Multi-task CNN architecture showing the shared feature extractor and dual classification heads.

As shown in Fig. 2, the input image passes through the four shared convolutional blocks (32, 64, 128 and 256 channels), and the resulting features feed two separate heads. In a single forward pass, the produce type head outputs the fruit class (e.g. "Banana") and the freshness state head outputs its condition (e.g. "Spoiled").

4.2 Training Performance & Test Metrics

Table 5 reports the training and validation progression across 15 epochs.

Table 5. Training and validation progression across 15 epochs.

Epoch	Training Loss	Produce Name Acc. (%)	Freshness Acc. (%)	Validation Loss	Val Produce Acc. (%)	Val Freshness Acc. (%)
1	1.842	48.2%	72.1%	1.210	65.4%	81.3%
5	0.812	76.5%	86.4%	0.652	81.2%	88.7%
10	0.385	88.9%	93.1%	0.398	89.1%	92.4%
15	0.194	94.8%	96.5%	0.281	92.3%	95.1%

As presented in Table 5, the model showed steady loss reduction and high accuracy convergence over 15 epochs. On the held-out test dataset, the joint exact-match rate (produce type and freshness both correct) reached 89.4%.

5. FIRMWARE, TELEMETRY PIPELINE & SERVER-SIDE INFERENCE

5.1 End-to-End Pipeline

The end-to-end architecture is illustrated in Fig. 3. At each acquisition cycle, the firmware reads the DHT22 and MQ135, captures a JPEG frame, and packages both into one multipart HTTP POST request [5]. A local inference server receives the request, decodes and resizes the image to 224×224×3, normalises it with the same statistics used during training, runs the multi-task CNN, applies the fusion rules and returns a JSON result to the node. Every record is stored for later analysis.

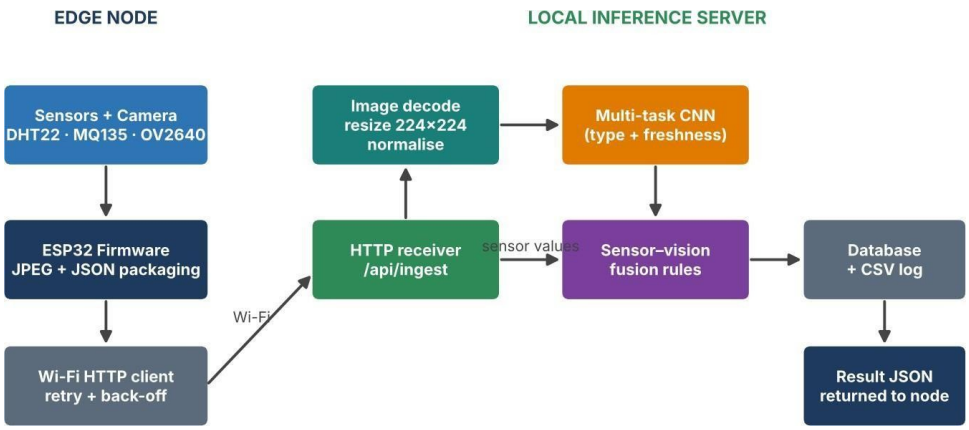


Fig. 3. End-to-end integration pipeline from the edge node to the local inference server.

As shown in Fig. 3, the edge node packages the sensor readings and the camera frame into one request and sends it over Wi-Fi with retry and back-off. On the server, the image is decoded, resized and normalised before the multi-task CNN, its output is combined with the sensor values in the fusion rules, and the result is stored in the database/CSV log and returned to the node as JSON.

5.2 Acquisition Cycle and Message Flow

Fig. 4 shows the sequence of one complete cycle. Pairing the image and the sensor readings in a single request guarantees that the CNN output and the gas value always refer to the same moment, which removes the need for timestamp matching on the server.

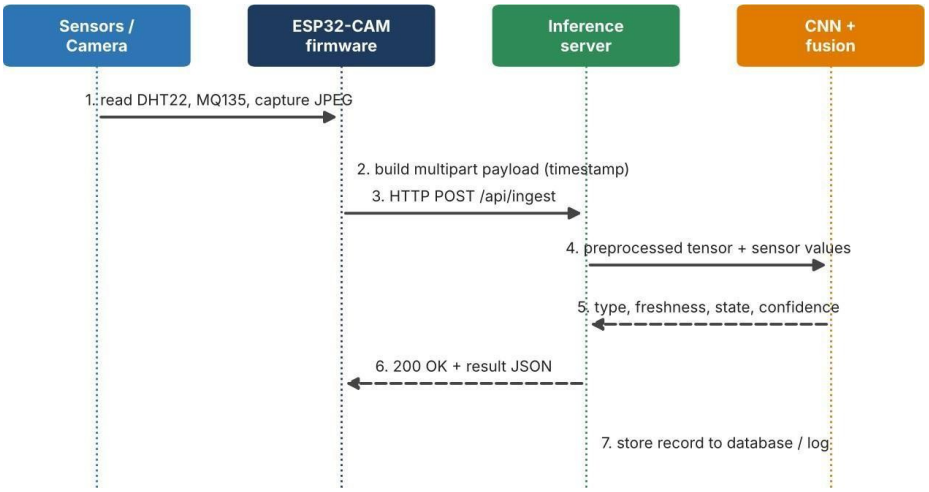


Fig. 4. Sequence of one acquisition and inference cycle.

As shown in Fig. 4, one cycle starts when the firmware reads the sensors, captures a JPEG, builds a timestamped multipart payload and posts it to /api/ingest. The server passes the preprocessed tensor and sensor values to the CNN and fusion stage, returns the produce type, freshness, state and confidence to the node with a 200 OK response, and stores the record.

5.3 Payload Format

Each request carries a JPEG part named image and a JSON part named telemetry, as in the example below. The server replies with the decision, which the firmware prints to the serial log.

```
// telemetry part (sent by ESP32-CAM)
{ "node": "esp32cam-01", "ts": 1792000000,
  "temp_c": 24.5, "hum_pct": 68.2, "gas_raw": 412 }

// response (returned by server)
{ "produce": "banana", "p_rotten": 0.12,
  "gas_ratio": 1.08, "state": "FRESH", "latency_ms": 0 }
```

5.4 Firmware Behaviour

- Non-blocking scheduling with millis(); the DHT22 is polled at a 2000 ms minimum interval to prevent NaN read errors [3].
- Camera frames are stored in the ESP32-CAM PSRAM [2] and sent as JPEG, which keeps the payload small compared with raw frames.
- Wi-Fi reconnection and request retries use exponential back-off; a watchdog restarts the node if the pipeline stalls.
- The MQ135 is given a warm-up period after power-on [4], and the first stable readings define the session baseline R0.

6. SENSOR-VISION FUSION LOGIC

The fusion stage was designed specifically to remove the main disadvantages identified in Section 2: blindness to produce type, static thresholds and missed internal decay. Fig. 5 shows the decision logic. The CNN supplies the produce type and the probability p(rotten). The MQ135 reading is converted to a baseline-relative ratio G = R/R0, which reduces the effect of sensor drift and of different starting conditions [9], [4]. The detected produce type selects a produce-specific gas limit, so that a banana is not judged by the same gas value as an orange.

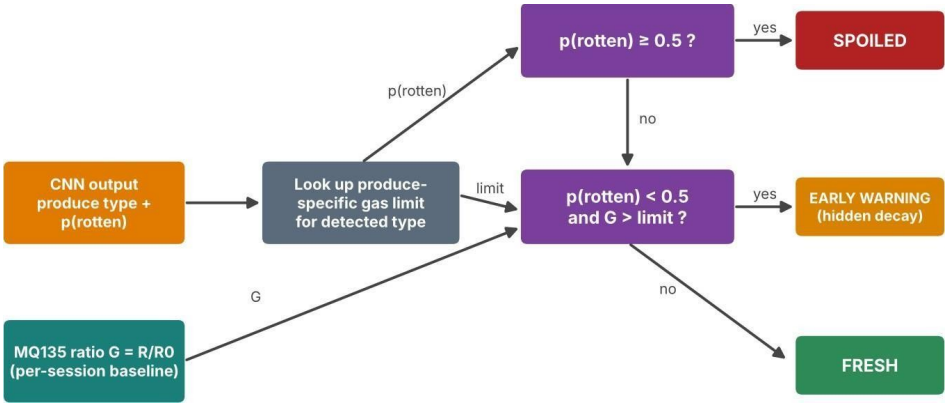


Fig. 5. Sensor-vision fusion decision logic.

As shown in Fig. 5, the CNN output is checked first: if p(rotten) ≥ 0.5, the fruit is labelled SPOILED. Otherwise, the gas ratio G is compared with the limit for the detected produce type; a value above the limit gives an EARLY WARNING of hidden decay, and a value within the limit gives FRESH.

Table 6. Fusion rules and output states.

Condition	Output State	Meaning
p(rotten) ≥ 0.5	SPOILED	Visual evidence of spoilage is sufficient on its own.
p(rotten) < 0.5 and G above the produce-specific limit	EARLY WARNING	Fruit looks acceptable but VOC release is abnormal; possible hidden decay.
p(rotten) < 0.5 and G within limit	FRESH	Both visual and gas evidence indicate acceptable condition.

As presented in Table 6, visual evidence of spoilage always takes priority, and the gas ratio is used to raise an early warning only when the fruit still looks acceptable. The produce-specific limits are tuned empirically from enclosure recordings of each fruit in its fresh state and are stored in a configuration file on the server, so they can be adjusted without reflashing the microcontroller.

7. INTEGRATION TESTING & RESULTS

Integration testing covered the communication path, the inference path and continuous operation. The model-level metrics were inherited from Report 5. The system-level measurements could not be recorded during Week 6 and are scheduled for Week 9, as no project work is conducted during Week 8. Table 7 summarises the test plan and the available results.

Table 7. Integration test summary.

Test	Criterion	Result
HTTP POST delivery (image + telemetry)	Request accepted (HTTP 200) with valid JSON	To be measured (Week 9)
Round-trip latency (capture to result)	Below 5 s per cycle	To be measured (Week 9)
Server inference time (CNN forward pass)	Below 1 s per image	To be measured (Week 9)
Continuous operation in enclosure	No unplanned reboot during test	To be measured (Week 9)
Wi-Fi drop recovery	Node reconnects and resumes automatically	To be measured (Week 9)
CNN joint exact-match (test set, Report 5)	Produce type and freshness both correct	89.4%
Validation produce / freshness accuracy (Report 5)	Epoch 15 validation	92.3% / 95.1%

As presented in Table 7, the model-level results carry over from Report 5, while each system-level test has a defined criterion and will be measured on the assembled prototype in Week 9. The pipeline was exercised end to end during integration: telemetry and images reach the server together, the model returns a decision for each frame, and the decision is logged for the multi-day experiments.

7.1 Expected Telemetry Trends

Fig. 6 shows the telemetry trend expected during the multi-day experiments as the fruit moves from fresh to spoiled, based on the behaviour of metal-oxide gas sensors [9], [4]. The MQ135 gas signal is expected to rise gradually from Day 0 and climb faster after the onset of spoilage around Day 2, while relative humidity inside the sealed enclosure increases more slowly. This rise in VOC concentration before and during visible spoilage is the behaviour that the gas-based early-warning rule in Section 6 relies on. Recorded data will replace this qualitative curve once the multi-day experiments are run.

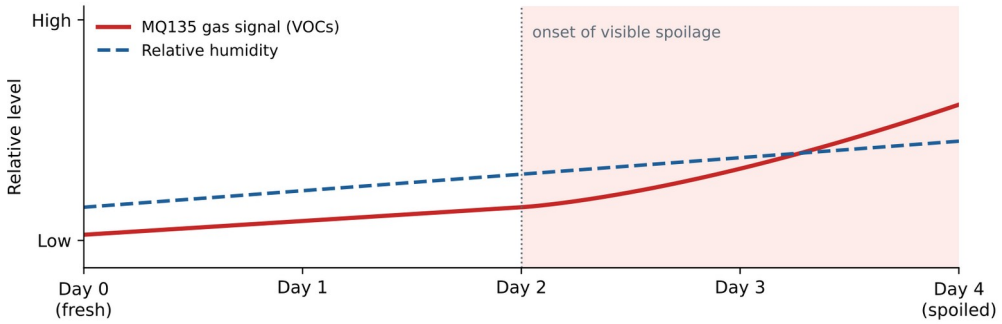


Fig. 6. Expected multi-day telemetry trend as fruit moves from fresh to spoiled (qualitative).

As shown in Fig. 6, the MQ135 gas signal is expected to increase slowly from Day 0, rise more steeply after spoilage begins around Day 2 and reach its highest level by Day 4, while the humidity curve increases only slightly over the same period.

8. TECHNICAL CHALLENGES, ROOT CAUSE ANALYSIS & APPLIED SOLUTIONS

The challenges below were met while integrating the hardware and the model. Each is listed with its root cause and the solution applied.

Table 8. Challenges encountered during integration.

Challenge	Root Cause	Applied Solution
Brownout resets	Simultaneous MQ135 heater current (~150 mA), camera initialisation and Wi-Fi transmission bursts.	Dedicated external 5 V regulated rail for the whole node [2], [4]; sensor and transmit tasks staggered in firmware.

Challenge	Root Cause	Applied Solution
Wi-Fi drops and HTTP timeouts	Signal fluctuation and a busy server during large uploads.	Exponential back-off retries, request timeout, automatic reconnection and a watchdog restart.
Large payloads and slow uploads	High-resolution frames increase upload time and memory use.	JPEG compression at moderate frame size [2]; final resize to 224×224 performed on the server.
Domain gap between training and live images	Public dataset images differ in colour balance and sharpness from OV2640 captures.	Fixed lighting and camera distance in the enclosure; identical preprocessing in training and inference [8], [13]; auto-exposure settled before capture.
MQ135 warm-up and drift	Metal-oxide heater needs stabilisation and the baseline changes with temperature and humidity.	Warm-up period before readings are used; baseline R0 recorded per session; ratio G used instead of raw values [9], [4].
Mismatch between image and sensor data	Sensor readings and frames sent separately would differ in time.	Single multipart request carrying image, readings and timestamp.

As presented in Table 8, each challenge was traced to a hardware, network or data cause, and each was resolved by a specific change in the power design, the firmware, the preprocessing or the payload format.

9. TEAM TASK DISTRIBUTION & PROGRESS ALIGNMENT

Table 9. Team task allocations and status alignment.

Workstream	Team Member(s)	Primary Responsibilities	Status
Hardware & Enclosure	Sara Saber, Abdelrahman Hisham	Maintained enclosure wiring and external power rail; verified sensor placement during integration.	Completed
Microcontroller Firmware	Mohamed Ahmed Galal	Implemented image + telemetry multipart POST, retry logic and non-blocking loops.	Completed
AI Model	Bashar Ahmed	Exported the trained model for the server and verified preprocessing consistency.	Completed
Server Integration	Mohamed Tamer, Mohamed Mostafa	Built the ingest endpoint, inference and fusion modules, and result logging.	Completed

As presented in Table 9, all four Week 6 workstreams completed their integration tasks, and each member's contribution maps to one stage of the end-to-end pipeline.

10. WEEK 6 OUTCOME & BRIEF DESCRIPTION OF REPORT 7

10.1 Week 6 Outcome

By the end of Week 6, the AI model and the sensing hardware were integrated into a single working pipeline: the ESP32-CAM transmits paired images and telemetry to a local inference server, the multi-task CNN returns produce type and freshness, and the fusion rules produce a final state that is logged for analysis. The disadvantages identified in the literature review were each addressed by a specific design decision documented in Table 1.

10.2 Brief Description of Report 7

Report 7 (Web Dashboard Development) will cover the local web dashboard and REST API for remote monitoring, the database that stores every reading, prediction and ground-truth label, and the alert logic that stabilises EARLY WARNING decisions. It will also document a bench review of the Week 6 setup and the corrective work it requires, and a revised schedule in which the system-level measurements and the multi-day experiments are carried out in Week 9 and INT8 quantization in Week 11.

11. REFERENCES

[1] R. Caruana, "Multitask learning," Mach. Learn., vol. 28, no. 1, pp. 41–75, 1997.

[2] Espressif Systems, "ESP32-CAM development board hardware reference and pinout guide," 2023.

[3] Aosong Electronics Co., Ltd., "Digital-output relative humidity & temperature sensor/module AM2302 (DHT22)," technical manual, 2021.

[4] Zhengzhou Winsen / Hanwei Electronics, "MQ-135 gas sensor technical data sheet."

[5] L. Masinter, "Returning values from forms: multipart/form-data," IETF RFC 7578, Jul. 2015.

[6] A. Bhattacharjee, N. A. Biswas, K. A. Shahriar, and K. B. Salim, "IoT-based cost-effective fruit quality monitoring system using electronic nose," arXiv:2512.08753, Dec. 2025.

[7] S. Cubero, N. Aleixos, E. Moltó, J. Gómez-Sanchis, and J. Blasco, "Advances in machine vision applications for automatic inspection and quality evaluation of fruits and vegetables," Food Bioprocess Technol., vol. 4, no. 4, pp. 487–504, 2011.

[8] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," Comput. Electron. Agric., vol. 147, pp. 70–90, 2018.

[9] J. W. Gardner and P. N. Bartlett, "A brief history of electronic noses," Sens. Actuators B, Chem., vol. 18, no. 1–3, pp. 211–220, 1994.

[10] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 2704–2713.

[11] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[12] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 32, 2019.

[13] R. Sriram, "Fruits fresh and rotten for classification," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/sriramr/fruits-fresh-and-rotten-for-classification